A Comparative Analysis of Deep Q-Network and Actor–Critic Reinforcement Learning Algorithms

Mykola Vaskevych

Department of Computer Science & Information Systems

Faculty of Science and Engineering

University of Limerick

FYP Report

Bachelor of Science in Computer Systems

2025 - 2026

1. Supervisor: Dr. J.J Collins
Associate Professor, Dept. of CSIS
University of Limerick
Ireland

2. Reviewer: Prof. Conor Ryan
Professor, Dept. of CSIS
University of Limerick
Ireland

Declaration

I herewith declare that I have produced this paper without the prohibited assistance of third parties and without making use of aids other than those specified; notions taken over directly or indirectly from other sources have been identified as such. This paper has not previously been presented in identical or similar form to any other Irish or foreign examination board.

The thesis work was conducted under the supervision of Dr. J.J. Collins at University of Limerick.

During the development of this project, I made use of AI tools — specifically Claude.ai and Claude Code by Anthropic. This included help with refining, such as making my explanations clearer and more structured, as well as assistance with code generation and debugging. Any AI-generated code was reviewed and understood by me before being used, and nothing was included that I could not explain or justify myself. AI was used as a practical tool to support my work, not to replace the thinking behind it.

Limerick, 2025–2026

Contents

List of Tables **v**

List of Figures **vii**

1 Introduction **1**

1.1 Overview 1

1.2 Research Questions 4

1.3 Methodology 4

1.3.1 Systematic Literature Review 4

1.3.2 Experimental Design 5

1.3.3 Data Collection 6

1.3.4 Data Analysis 7

1.3.5 Contributions 7

1.4 Project Plan 8

1.5 Overview of the Report 8

2 Literature Review **11**

2.1 Artificial Intelligence and Machine Learning 11

2.2 Foundations of Reinforcement Learning 13

2.3 Action Selection and the k-Armed Bandit Problem 15

2.4 Markov Decision Process 16

2.5 Value Based Reinforcement Learning Algorithms 19

2.5.1 Dynamic Programming 19

2.5.2 Monte Carlo Methods 20

2.5.3 Temporal Difference Learning 21

2.6 From TD to DQN 23
2.6.1 Convolution Neural Networks and Deep Q Networks 23
2.6.2 What is CNN 24
2.6.3 DQN 25
2.7 Policy Gradients and Actor-Critic Methods 27
2.7.1 REINFORCE Algorithm 29
2.7.2 REINFORCE with Baseline 31
2.7.2.1 Episodic REINFORCE with Baseline 32
2.7.3 Actor – Critic Methods 33
2.7.3.1 Advantages of Actor-Critic 34
2.7.4 Advantage Actor-Critic (A2C) 35
2.7.5 Proximal Policy Optimisation (PPO) 35
2.8 Experimental Frameworks and Tooling 35
2.8.1 Gymnasium 35
2.8.2 Stable-Baselines3 36
2.8.3 Stable-Baselines3 Contrib and Extensions 36
2.8.4 Reproducibility Considerations 37
2.9 Evaluation Framework 37
2.9.1 Data Produced by RL Interaction 37
2.9.2 Evaluation Scores: Definitions and Notation 38
2.9.3 Uncertainty Quantification: Stratified Bootstrap 38
2.9.4 Per-Environment Metrics (Primary) 38
2.9.5 Cross-Environment Aggregate Metrics (Secondary) 39
2.10 Conclusion 39

3 Evaluation **41**
3.1 Research Question 41
3.2 Synthesis of Results 42
3.2.1 Experiment 1: Classic-Control Benchmark 42
3.2.2 Experiment 2: Atari Benchmark 42
3.2.3 Experiment 3: Extended Atari Benchmark 42
3.3 Answer to the Research Question 43
3.4 Threats to Validity 43

4 Discussions & Conclusions **45**
4.1 Future Work 45

References **47**

List of Tables

List of Figures

2.1 Agent–environment interaction in a Markov Decision Process. . . 18
2.2 Pacman Game Example. Source: unknown (2023) 24
2.3 Inside a convolutional network 25
2.4 DQN Architecture on Atari Breakout Mnih et al. (2015) 26
2.5 Adding a baseline to REINFORCE can make it learn much faster. The step size used for plain REINFORCE is that at which it performs best. Source: Sutton and Barto (2018). 32

Chapter 1

Introduction

1.1 Overview

This Final Year Project compares two families of deep reinforcement learning algorithms: value-based methods, which learn by estimating the value of situations, and actor-critic methods, which learn by directly improving the strategy that works. Three experiments of increasing scope and depth test both families under identical conditions — same framework, same default settings, no per-algorithm tuning — so that differences in results reflect genuine algorithmic character rather than configuration effort.

The results show that actor-critic methods are the more reliable default: they produce strong, consistent results across all tested settings with no special setup. Value-based methods are not weaker — they are more demanding. They require more training time, more careful configuration, and greater domain knowledge to reach their potential. This distinction matters for practice: the right question is not which family is better, but which is appropriate given the available budget and expertise.

A secondary finding is methodological: from the same data, different evaluation metrics can produce opposite conclusions about which algorithm family wins. This project demonstrates that a single aggregate number is insufficient — and potentially misleading — when comparing reinforcement learning algorithms.

Reinforcement learning is learning what to do — how to map situations to actions — so as to maximize a numerical reward signal. The learner is not

told which actions to take, but instead must discover which actions yield the most reward by trying them. In the most interesting and challenging cases, actions may affect not only the immediate reward but also the next situation and, through that, all subsequent rewards. These two characteristics—trial-and-error search and delayed reward—are the two most important distinguishing features of reinforcement learning. Sutton and Barto (2018)

Learning how to play chess can be a good example of Reinforcement learning, by making discrete steps in the environment, which are called actions (moving a piece) and getting back a reward (how good your current position), we can determine how good our move was, and vice versa, learn to make better moves. The difference from other methods unique to RL is that we give a score to the actions taken rather than saying correct/incorrect as is done in supervised learning. In RL, you need to balance between exploitation of already learned actions with good reward and exploration to potentially find even better actions to exploit for better reward in the long-term goal.

Reinforcement Learning is designed to solve problems that can be modeled as sequential decision processes. Barto et al. (1989) Learning to play chess is a sequential decision process, as every action taken by the player (agent) transitions the environment (chess board) to a new state. For example, moving the knight to the right cell may lead to checkmate of the enemy's king and winning or open an opportunity for your opponent to attack you instead, which may lead to losing the game.

The search space for the game of Connect 4 is 10^{12} and for chess is 10^{40}. It is difficult to enumerate each board in such large search spaces using brute force approach such as breadth first or depth first search algorithms. But this was achieved by IBM's Deep Blue super computer beat the chess world champion Garry Kasparov back in 1996. However, the search space for other games such as Go is significantly larger and brute search approaches do not work. Instead one has to use function approximators and deep learning architectures in particular given the recent success of Convolutional Neural Networks (CNNs) in computer vision.

Convolutional Neural Networks (CNNs) are a type of neural network designed specifically for data that comes in grid-like formats, such as images. Instead of

treating every input value independently, a CNN works with small local regions of the image and learns filters that can detect useful patterns. These filters slide across the input, allowing the network to find the same pattern in different locations. This idea of local connections and shared weights is one of the core principles of CNNs, along with pooling and stacking multiple layers to build deeper feature representations Lecun et al. (2015)

There are two major families of algorithms in the RL family tree. One branch is Value Functions which estimate the value of the likely long term reward from the current position or from slecting an action in the current position. Typical of thiese is Q Learning Method, and its implementation on deep learning architecture called Deep Q Network (DQN). This was used by DeepMind with layers of tiled convolutional filters to mimic the effects of receptive fields. Reinforcement learning is unstable or divergent when a nonlinear function approximator such as a neural network is used to represent Q. This instability comes from the correlations present in the sequence of observations, the fact that small updates to Q may significantly change the policy of the agent and the data distribution, and the correlations between Q and the target values. The method can be used for stochastic search in various domains and applications.Li (2023) Matzliach et al. (2022)

The other major branch uses the Policy Gradient approach. Instead of learning value functions, this group of algorithms parameterizes the policy directly and returns probabilities for each possible action in the current state. Along with other distinguishing features that will be discussed later, Policy Gradient methods allow us to train networks in spaces which are not handled well by value-based algorithms—such as stochastic environments or continuous action spaces, where there is no certainty of which action is best, where functions like argmax do not work, or where the number of possible actions is enormous.

For Prototypes developed in this Report I used Gymnasium which is a maintained fork of OpenAI Gym and is now the recommended standard for developing and evaluating reinforcement learning agents. Gymnasium is described as "an API standard for reinforcement learning with a diverse collection of reference

environments" and provides a simple, pythonic interface capable of representing general reinforcement learning problems.

1.2 Research Questions

RQ: How does the performance of value-based Deep Q-Network agents compare to actor-critic agents (specifically PPO and A2C) in discrete action space environments, including their sample efficiency and training consistency during training?

These algorithms will be compared based on several criteria: **convergence speed** (number of episodes to reach the threshold) **sample efficiency** (performance per interaction) **training stability** (reward variance).

These comparison criteria will serve as the basis for evaluating the trade-off between the fast, off-policy learning of value-based methods versus the stable, on-policy learning of actor-critic methods.

Secondary Objective: Achieve a solid understanding of the foundations of artificial intelligence, machine learning, and deep reinforcement learning.

this chpater needs check/update

1.3 Methodology

This section outlines the methodologies followed to ensure the successful execution of this project.

1.3.1 Systematic Literature Review

To ensure the prototype development and experimentation tasks were grounded in existing research, a systematic literature review approximation of artificial intelligence, machine learning, reinforcement learning, value-based, and policy based reinforcement learning relevant texts and research papers was first conducted. Google Scholar, arXiv, and IEEE Xplore were electronic databases employed during the search process. Search terms used were "Artificial Intelligence", "Machine Learning", "Reinforcement Learning", "Deep Reinforcement Learning", "Deep

Q-Network", "Actor-Critic", "Policy Gradient", "A2C", and "PPO". The inclusion criteria were articles and research papers that discussed at least one fundamental machine learning or reinforcement learning concept, or value-based or actor-critic reinforcement learning algorithms examined in this final year project. Educational video content and other online resources, such as framework documentation, were also queried where applicable. "Reinforcement Learning: An Introduction" by Andrew Barto and Richard S. Sutton, and "Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow" by Aurélien Géron were two texts heavily relied upon when gaining knowledge of machine learning and reinforcement learning fundamentals.

The systematic literature review approximation can be broken down into five topics, as outlined below:

1. The Foundations of Artificial Intelligence and Machine Learning: History, Semantics, PSSH, POE Model, Machine Learning Approaches.

2. The Fundamental Principles and Approaches of Reinforcement Learning: Terminology, Exploitation vs Exploration, The Credit Assignment Problem, MDPs, The Bellman Equation, Dynamic Programming, GPI, Monte Carlo Algorithm, Temporal Difference Learning, Q-learning.

3. Deep Learning: Threshold Logic Unit, Perceptron, Multilayer Perceptron, Backpropagation, Hyperparameters, Gradient Instability, Batch Normalization, Convolutional Neural Networks.

4. Value-Based and Actor-Critic Reinforcement Learning Algorithms: Rationale, Deep Q-learning, Policy Gradients, Actor-Critic Methods, Advantage Estimation, On-Policy vs Off-Policy Learning, Training Stability.

5. Atari Breakout and Statistical Significance: OpenAI, History, Rationale.

1.3.2 Experimental Design

While studies comparing these algorithm families exist (De La Fuente and Guerra), they typically involve per-algorithm hyperparameter tuning, making it difficult to separate algorithmic capability from configuration effort. This project takes

a different approach: Stable-Baselines3 default hyperparameters are used for every algorithm across all three experiments, with no tuning of any kind. The importance of this distinction is illustrated by Mnih et al. (2015), who acknowledge that their published DQN results relied on hyperparameters "selected by performing an informal search," noting that "it is conceivable that even better results could be obtained by systematically tuning the hyperparameter values." All results in this thesis reflect what each algorithm delivers out of the box — an intentional choice to measure inherent algorithmic behaviour rather than tuning effort. Moreover, implementing both algorithm families facilitated the validation of existing research and the expansion on recent findings. Establishing a basis for future research and fostering transparency were other motivating factors for prototype development.

In addition to discussing the main implementation components that comprise the DQN and Actor-Critic architectures, Chapter 3 provides a comprehensive overview of the high-level architectural decisions, equipment employed, and problems encountered during experimentation. Chapter 3 can be broken down into four topics:

1. High Level Architecture Decisions: Framework, Program Structure, Implementation Medium.

2. Code Implementation: Atari Breakout Environment, DQN and Actor-Critic Architectures, Recording Training Progress.

3. Equipment Employed.

4. Problems Encountered: GPU Driver Compatibility, Memory Leak.

1.3.3 Data Collection

Data collection followed the same protocol across all three experiments. During training, performance was measured via periodic frozen-policy evaluations rather than live training rewards, since training rewards are algorithm-dependent and not a stable basis for comparison. At fixed step intervals, each agent was evaluated

for a fixed number of episodes in a separate environment with no exploration noise, and the mean episodic return was recorded.

Across all experiments, training used Stable-Baselines3 (Section 2.8) with its built-in `EvalCallback`, which handles evaluation scheduling and logging automatically. Final performance was assessed with a dedicated post-training evaluation of 50 episodes per seed. All seeds were derived deterministically from a single master seed per experiment using NumPy's `SeedSequence`, ensuring reproducibility. Full details of evaluation frequency, episode counts, and normalisation are defined in Section 2.9.

1.3.4 Data Analysis

Results were analysed using the `rliable` statistical evaluation framework (Agarwal et al.) (Section 2.9). For each experiment, per-environment learning curves show performance over the training budget across all seeds, giving a picture of both speed and consistency. Final performance is summarised using interquartile mean (IQM) and pairwise probability of improvement, each reported with 95% stratified bootstrap confidence intervals. Cross-environment aggregates are treated as descriptive summaries rather than definitive rankings, given the small number of environments per experiment. Full metric definitions and the normalisation procedure are provided in Section 2.9.

1.3.5 Contributions

This project makes three contributions to the empirical study of deep reinforcement learning.

1. A three-experiment benchmark comparing value-based and actor-critic algorithm families, using a single shared framework with default hyperparameters throughout (see Section 2.8). Over 80 training runs were conducted across classic-control and Atari environments, with results reproducible and consistent across seeds.

2. Application of the `rliable` statistical evaluation framework (Section 2.9) to a student-scale RL study, reporting interquartile mean, probability of

improvement, and bootstrap confidence intervals rather than mean scores alone.

3. An observation that cross-environment aggregation metric choice affects the conclusion of the comparison: the same data produces different aggregate rankings under IQM versus pairwise probability of improvement. This is not a flaw in the methodology but a finding about the limits of single-number RL benchmarks.

1.4 Project Plan

The first semester was spent building the theoretical foundation. This involved working through the literature on reinforcement learning fundamentals — MDPs, Bellman equations, Q-learning, policy gradients, and actor-critic methods — as well as implementing several small-scale environments (Cliff Walking, Windy Gridworld, Random Walk) to build intuition before tackling anything larger. The semester ended with a working DQN agent on Atari Breakout as a proof of concept.

The second semester moved into the main comparison. Three experiments were run in sequence, each extending the scope of the previous one. Experiment 1 covered five algorithms on classic-control environments using MLP policies. Experiment 2 narrowed the focus to DQN vs PPO on Atari with CNN policies at a 5M-step budget. Experiment 3 extended that to 20M steps and added Seaquest as a third environment. The results of all three are reported in Chapters 4, 5, and 6 respectively.

1.5 Overview of the Report

Chapter 2 reviews the theoretical foundations of reinforcement learning — the two algorithm families, the key algorithms from each, and the tools and evaluation framework used throughout. Chapter 3 presents the prototype work from the first semester: a DQN implementation on Atari Breakout that established the technical foundation for the experiments. The project GitHub repository contains the

full source code for all experiments, commit history, and development statistics, and is linked in Chapter 3.

The three experiment chapters were not designed upfront as a fixed plan — each arose from the findings and open questions of the previous one. Chapter 4 evaluates five algorithms across both families on classic-control environments. Algorithm and environment descriptions are provided in Chapter 2. Chapter 5 narrows the comparison to one representative per family and scales to pixel-based Atari games. Chapter 6 extends that comparison to a longer training budget and a wider set of environments. Each chapter follows the same structure: setup, hypotheses, results, key findings, threats to validity, and discussion.

Chapter 7 synthesises the findings across all three experiments and provides a direct answer to the research question. Chapter 8 closes the report: it restates the conclusions plainly, reflects on what made this project difficult, and outlines directions for future work.

Chapter 2

Literature Review

2.1 Artificial Intelligence and Machine Learning

Artificial Intelligence (AI) is a broad research field focused on designing and studying computational systems capable of performing tasks traditionally associated with human intelligence, such as reasoning, learning, problem-solving, perception, planning, and decision-making Russell and Norvig (2021).

Rather than relying on a single methodology, AI involves all sorts of approaches that allow machines to perceive their environment and select actions that increase the probability of achieving defined objectives.

Since its formal recognition as an academic discipline in the mid-20th century, AI research has pursued objectives such as knowledge representation, logical reasoning, natural language understanding, visual perception, and autonomous control Russell and Norvig (2021). Achieving these objectives has required integrating diverse techniques, including symbolic logic, search and optimization methods, probabilistic modeling, artificial neural networks, and insights from related fields such as psychology, neuroscience, linguistics, and economics. Consequently, AI is best characterized as a multidisciplinary research area unified by the pursuit of intelligent behavior.

From the AI sphere, the Machine Learning (ML) has emerged as one of the most significant paradigms. Rather than relying on explicitly hand-crafted rules like GOFAI, machine learning focuses on algorithms that improve performance on specific tasks through experience, without human-defined if-else rules. This

shift has been critical for solving problems that involve many variables, where manual rule specification is impractical. Advances in computing power and data availability have enabled learning-based approaches to consistently improve and, in many cases, surpass classical symbolic reasoning systems across diverse application domains.

Machine learning methods are typically classified into three categories, which are distinguished by the type of learning signals provided to the agent:

- **Supervised Learning**, in which an algorithm learns a mapping from inputs to outputs using labeled data and optimizes a predefined loss function.
- **Unsupervised Learning**, where the objective is to discover latent structure in unlabeled data, such as clusters or low-dimensional representations.
- **Reinforcement Learning (RL)**, where an agent learns through direct interaction with an environment and receives only scalar reward signals as feedback.

Reinforcement learning differs fundamentally from supervised and unsupervised learning. RL is inherently interactive and sequential. The data distribution during training is not fixed; the agent's actions influence it. Each decision impacts immediate rewards, future states, and the data available for future learning. This feedback loop introduces challenges unique to RL, including delayed rewards, non-stationarity, and instability.

The development of AI has been marked by alternating periods of optimism and stagnation. Early symbolic systems achieved success in restricted domains but were unable to address real-world complexity, leading to periods of reduced funding and enthusiasm known as AI winters Crevier (1993); National Research Council (1999); Russell and Norvig (2021). Substantial progress resumed with the advent of data-driven learning, particularly after 2012, when advances in computational hardware and neural network architectures enabled deep learning methods to surpass earlier AI approaches across a wide range of tasks Goldman (2022); McKinsey & Company (2018); Russell and Norvig (2021).

Many successful AI techniques become so integrated into everyday systems that they are no longer identified as AI CNN (2006); Kaplan and Haenlein (2019).

This underscores that AI is defined less by specific algorithms than by its evolving capacity to address tasks once considered exclusive to human intelligence.

Within this context, reinforcement learning occupies a central position in AI research. By formally addressing decision-making under uncertainty and delayed feedback, RL provides a principled framework for learning control policies in complex environments. The following sections introduce the fundamental concepts of reinforcement learning and clarify its distinctions from other machine learning paradigms.

2.2 Foundations of Reinforcement Learning

Reinforcement Learning is a sequential decision-making learning process through trial and error. It is one of the three main types of machine learning, along with supervised and unsupervised learning. In supervised learning, correct answers are given, but in reinforcement learning, agents learn what works by trying different actions and learning from the results. Let me make an example on chess game. Think of chess engines like Stockfish - they observe the board, make a move, get feedback from winning or losing, and learn optimal strategies through millions of games.

In other words, RL learns by iteration a agent–enviroment loop. At each point of time t, the agent observes the current state S_t of the environment and makes an action A_t according to its policy.

In response, the environment transitions to a new state S_{t+1} and sends back a reward R_{t+1} to determine how good the taken action was.

This process represents a sequence of states, actions, and rewards that constitute the agent's experience.

The objective of the agent is to learn a policy that maximizes the expected cumulative reward over time rather than immediate reward alone Barto et al. (1989). Where policy is the set of rules that the algorithm follows.

A challenge of reinforcement learning is the presence of **delayed rewards**. The consequences of an action may only become observable after many following interactions, making it difficult to directly attribute outcomes to individual decisions. This difficulty is known as the **reward assignment problem** and is

an important problem in the design of reinforcement learning algorithms Sutton and Barto (2018). For example, in long-term planning tasks, early actions may strongly influence final results even when they yield no immediate reward. So, reinforcement learning algorithms must reason about long-term consequences and juggle short-term and long-term rewards.

Formally, reinforcement learning problems are commonly modeled as **Markov decision processes (MDPs)**, which provide a mathematical model for sequential decision-making in uncertainty. An MDP is defined by a set of states, a set of actions, possibilities of state changes, a reward function, and a discount factor. Classical dynamic programming methods assume full knowledge of these components; reinforcement learning algorithms typically operate without access to an explicit model of the environment and instead learn from sampled experience Kaelbling et al. (1996).

Another core element of reinforcement learning is the requirement to balance **exploration** and **exploitation**. To improve performance, an agent must exploit actions that are known to get high rewards. However, discovering such actions requires exploration of actions with unknown reward. The search for an effective compromise between exploration vs exploitation is a central theme in reinforcement learning research and has been studied extensively in the context of multi-armed bandit problems and finite Markov decision processes Burnetas and Katehakis (1997). Inadequate exploration can result in non-optimal policies, while too much exploration may prevent convergence to effective behavior.

Reinforcement learning differs from other optimization-based approaches, such as evolutionary algorithms, in how it uses experience. Evolutionary methods usually evaluate complete policies as indivisible units, whereas reinforcement learning algorithms learn structured representations, such as value functions or parameterized policies, that assign credit to individual states and actions Sutton and Barto (2018). This structure makes more efficient reuse of experience across similar situations and supports generalization to previously unseen states.

2.3 Action Selection and the k-Armed Bandit Problem

Consider the following learning problem. You are repeatedly faced with a choice among k different options or actions. After each choice, you receive a numerical reward chosen from a stationary probability distribution that depends on the action you selected. Your objective is to maximize the expected total reward over some time period, for example, over 1000 action selections or time steps. This is the original form of the k-armed bandit problem, so named by analogy to a slot machine, or "one-armed bandit," except that it has k levers instead of one. Each action selection is like a play of one of the slot machine's levers, and the rewards are the payoffs for hitting the jackpot. Through repeated action selections, you are to maximize your winnings by concentrating your actions on the best levers. Sutton and Barto (2018).

To learn the best actions, the agent keeps the value of each action, denoted by $Q_t(a)$, which represents the expected reward obtained by selecting action a. A common estimation approach is the **sample-average method**, defined as:

$$Q_t(a) = \frac{\sum_{i=1}^{t-1} R_i \cdot \mathbb{1}_{A_i=a}}{\sum_{i=1}^{t-1} \mathbb{1}_{A_i=a}},$$

where $\mathbb{1}_{A_i=a}$ is the indicator function that equals 1 when action a was selected at time i, and 0 otherwise. It converges to the true action value under the belief of fixed reward allocation.

To avoid the high cost of memory for storing all the information that the agent gathers, the sample average update is used, which can be formulated as:

$$Q_{n+1} = Q_n + \alpha \left[R_n - Q_n\right],$$

where α is a step-size parameter. The quantity $[R_n - Q_n]$ represents the estimation error, between the reward and the current value estimate. This update follows the general pattern:

$$\text{New Estimate} \leftarrow \text{Old Estimate} + \alpha(\text{Target} - \text{Old Estimate}),$$

which is a common way across a wide range of reinforcement learning algorithms.

The k-armed bandit problem highlights the fundamental **exploration–exploitation issue**.

As a real person on slot machines, an agent wants to get higher rewards possible (exploit), but also hopes to get a more profitable way by exploring other actions whose value is uncertain.

While this trade-off becomes more complex in full reinforcement learning problems involving state transitions, its core nature is already present in the example of the K-bandit scenario. As such, the k-armed bandit provides a foundational testbed for developing and analyzing exploration strategies Burnetas and Katehakis (1997).

Despite the bandit problem omitting many aspects of MDP, it still plays a significant role in the theory of reinforcement learning. Many similar mechanisms of action selection, with exploration strategies involved, have been developed for bandits and generalize directly to more complex environments, making the k-armed bandit an important stage of progress towards understanding the full Markov decision process.

2.4 Markov Decision Process

The k-armed bandit problem ignores an essential feature of most real-world decision-making problems that: **actions influence future states**. In the real world, actions have consequences in the future, and while some actions may be optimal right now, they might potentially harm you in the future, or just be suboptimal. Continuing with my favorite example of the chess game, where you may capture a queen of the opponent but miss a 3-move checkmate, and lose, or to put it more formally, where capturing a valuable piece may immediately increase material advantage, yet expose the player to a forced checkmate shortly afterward. Reinforcement learning algorithms must therefore reason about *long-term cumulative reward*, rather than immediate reward.

This category of problems, where future matters, is what **Markov Decision Process (MDP)** is about. An MDP provides a framework described mathematically for sequential decision-making processes and is defined by the Markov property: the future is conditionally independent of the past given the present state.

Formally,

$$P(S_{t+1} \mid S_t, A_t) = P(S_{t+1} \mid S_1, A_1, \ldots, S_t, A_t).$$

This means that the current state S_t must contain all information necessary to predict future state transitions. If the data necessary for prediction is missing, then we can say that MP is violated.

An MDP is described by the following elements:

- a set of states $\mathcal{S}$,
- a set of actions $\mathcal{A}$,
- a transition probability function $P(s' \mid s, a)$,
- a reward function $R(s, a, s')$,
- and a discount factor $\gamma \in [0, 1]$ (more in the future, less reward meaning for the present).

At each time step t, the agent observes the current state S_t, selects an action A_t, and the environment reacts by moving the agent to a new state S_{t+1} while ejecting a numerical reward R_{t+1}. This interaction loop, illustrated in Figure 2.1, forms the core of reinforcement learning.

The goal of an agent is to learn an optimal strategy how to get best rewards(policy). The core of this goal is a concept of the **action-value function**, or **Q-value**. The Q-value $Q(s, a)$ represents the expected return of action a in specific state s. Basically, it responds to the question:

"How good is this action in this situation"

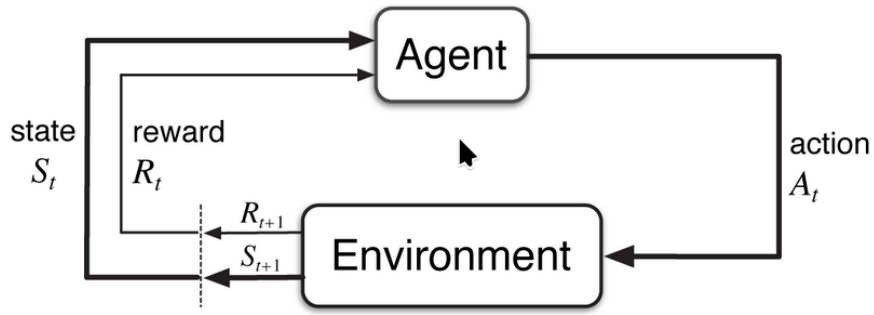

Figure 2.1: Agent–environment interaction in a Markov Decision Process.

To reason about long-term consequences, reinforcement learning introduces the concept of **return**. The return G_t is defined as the total accumulated reward an agent receives from time step t onward. The return G_t is defined as the discounted sum of future rewards:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + ... = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

where $\gamma \in [0, 1]$ is the **discount factor** that determines the present value of future rewards.

These values are related through the **Bellman principle**, which states that the value of a state or state–action pair depends on the immediate reward and the values of future states. In theory, this principle defines the optimal solution exactly. However, computing it directly requires full knowledge of the environment and repeated recursive updates, which is computationally infeasible for most real-world problems. Approximation of Q-values happens from experience of an agent, and most methods of approximation are splited into 3 categories: **dynamic programming**, **Monte Carlo methods**, and **temporal-difference learning**. These approaches differ in how they use experience and feedback, and they form the foundation of the algorithms discussed in the following sections.

2.5 Value Based Reinforcement Learning Algorithms

2.5.1 Dynamic Programming

Dynamic Programming (DP) is a class of algorithms used to compute optimal value functions and policies when the **complete environment model is known**. For DP to be applicable, the problem must be formulated as a finite Markov Decision Process (MDP), and all states, actions, rewards, and transition dynamics $p(s', r \mid s, a)$ must be available.

Rather than solving the Bellman optimality equations directly, Dynamic Programming methods compute value functions iteratively using repeated updates of the form:

$$\text{Old Value} \Rightarrow \text{Update Rule} \Rightarrow \text{New Value}$$

These updates are applied repeatedly until the values converge.

Two main Dynamic Programming methods are commonly used:

- **Policy Iteration**: alternates between policy evaluation and policy improvement. It typically converges in fewer iterations, but each iteration is computationally expensive.

- **Value Iteration**: combines policy evaluation and improvement into a single update. It requires more iterations, but each update is computationally cheaper.

Both methods are guaranteed to converge to the optimal policy and often converge quickly in small problems. However, Dynamic Programming is not very practical in real-world applications because it requires exact knowledge of the environment dynamics and becomes computationally intense for large state spaces.

2.5.2 Monte Carlo Methods

Monte Carlo methods require only experience. Sutton and Barto (2018), without needing a model of the environment, its main difference from Dynamic Programing which requires full environment dynamics $p(s', r \mid s, a)$.

Monte Carlo methods work only for **episodic tasks**, which means tasks that have an ending/finish.

Sutton and Barto explicitly note that "to ensure that well-defined returns are available, here we define Monte Carlo methods only for episodic tasks" Sutton and Barto (2018). Estimated values are being updated only at the end of the episode.

To make an estimation of the state -value function $v_\pi(s)$, an agent follows a policy π and samples many episodes. Every time the agent is in state s, it records the return G_t, which is a cumulative reward from that time step until the end of the episode, by averaging these returns, we get an estimated value of this state. By the law of large numbers, sooner or later, this average would converge to the true expected value.

Blackjack Example As an example of MC, let's overview a classical casino game, "Blackjack". Playing blackjack is naturally formulated as an episodic finite MDP. Sutton and Barto (2018)

Each game represents one episode, with rewards of $+1$, 0, or -1 for a win, draw, or loss, respectively. All middle rewards are zero, and discounting is not required ($\gamma = 1$), meaning the return equals the final reward.

Each state is defined as a sum of the values of players' cards, if the dealer shows his cards, and if the agent has a usable ace. By iterating game episodes over and over under a fixed policy, MC would produce an accurate approximation of the state-value function. This would highlight an important advantage of Monte Carlo methods: while calculating the exact transition probabilities is difficult, the process of generating sample episodes to get probability approximation is straightforward.

Monte Carlo methods learn directly from experience and do not require a model of the environment. They do not bootstrap, meaning that value estimates

are updated using complete returns rather than other estimates. MC methods are simple and intuitive, but they must wait until the end of an episode to update, which can result in high variance and slow learning.

2.5.3 Temporal Difference Learning

Temporal-Difference learning or TD can be seen as a combination of Monte Carlo (MC) methods and Dynamic Programming (DP) Sutton and Barto (2018). Like Monte Carlo methods, TD learning does not require a model of the environment and learns directly from experience. Like Dynamic Programming, TD methods update value estimates using other learned estimates, a process known as **bootstrapping**.

The key difference between TD and Monte Carlo methods is **when** updates are made. Monte Carlo methods must wait until the end of an episode to compute the full return and update values. In contrast, TD methods update estimates after each time step (it bootstraps), using the observed reward and the estimated value of the next state. This allows TD methods to learn *online*, without waiting for the final outcome.

TD Prediction The simplest TD method is known as **TD(0)**, or one-step TD learning. The update rule for estimating the state-value function is given by:

$$V(S_t) \leftarrow V(S_t) + \alpha \left[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)\right]$$

where α is the learning rate and γ is the discount factor.

The expression inside the brackets,

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t),$$

is called the **TD error**. It measures how different the current estimate is from a better estimate based on the next observed transition. This error signal plays a central role in almost all reinforcement learning algorithms.

Unlike Monte Carlo methods, TD methods do not require episodes to terminate and can therefore be applied to **continuing tasks**. This makes TD learning applicable to a wider range of real-world problems.

n-step TD and the MC–TD Connection TD learning can be generalized to n**-step TD methods**, which update values using n future rewards:

$$V(S_t) \leftarrow V(S_t) + \alpha \left[G_{t:t+n} - V(S_t) \right]$$

where

$$G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n}).$$

These methods form a bridge between TD and Monte Carlo learning:

- $n = 1$ corresponds to TD(0),
- $n \to \infty$ (or n equal to episode length) corresponds to Monte Carlo learning.

Advantages of TD Learning TD methods offer several important advantages over Monte Carlo and Dynamic Programming approaches:

- Updates are performed after every step rather than once per episode.
- TD methods can be applied to non-episodic (continuing) tasks.
- Previously learned estimates can be reused immediately through bootstrapping.
- In practice, TD methods often converge faster than Monte Carlo methods on stochastic problems Sutton and Barto (2018).

These advantages are demonstrated in classical examples such as the random walk problem, where TD(0) is shown to achieve lower prediction error than constant-step-size Monte Carlo methods.

TD Control Methods TD learning can also be applied to the control problem by learning action-value functions $Q(s, a)$.

Sarsa is an on-policy TD control algorithm that updates values using the actual action taken by the agent:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t) \right].$$

Because Sarsa accounts for the agent's exploration behavior, it often learns safer policies.

Q-learning is an off-policy TD control algorithm that directly learns the optimal action-value function:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right].$$

Q-learning learns the optimal policy independently of the agent's behavior, but its online performance can be unstable due to exploration.

Expected Sarsa replaces the maximum operator with an expectation over the policy, reducing variance and often improving performance in practice.

2.6 From TD to DQN

Within the TD framework, **Q-Learning** stands out as an off-policy algorithm that learns the optimal action-value function $Q^*(s, a)$ independently of the agent's behavior. The Q-learning update is:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$$

This specific equation is the direct mathematical ancestor of the **Deep Q-Network (DQN)**. By replacing the lookup table of $Q(s, a)$ with a deep neural network, DQN allowed RL to scale to high-dimensional environments like Atari games, which is a core focus of this project.

2.6.1 Convolution Neural Networks and Deep Q Networks

The main problem with Q-learning is that it does not scale well to large (or even medium) MDPs with many states and actions. For example, suppose you wanted to use Q-learning to train an agent to play Ms. Pac-Man (see Figure 2.2). There are about 150 pellets that Ms. Pac-Man can eat, each of which can be present or absent (i.e., already eaten). So, the number of possible states is greater than $2150 \approx 1045$. And if you add all the possible combinations of positions for all the ghosts and Ms. Pac-Man, the number of possible states becomes larger than the

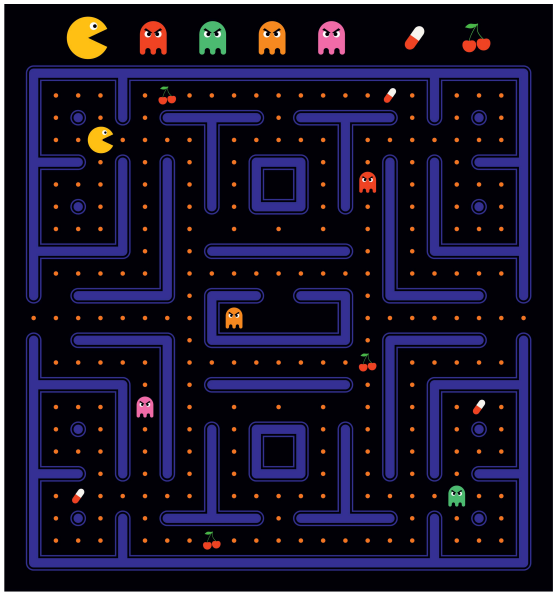

Figure 2.2: Pacman Game Example. Source: unknown (2023)

number of atoms in our planet, so there's absolutely no way you can keep track of an estimate for every single Q-value. Géron (2022)

The Solution is was suggested in 2013 by DeepMind when they published paper about DQN. Mnih et al. (2015). What they did is cobmined CNN with Q-learning and called it DQN.

2.6.2 What is CNN

Convolutional Neural Networks (CNNs) are a type of neural network designed specifically for data that comes in grid-like formats, such as images. Instead of treating every input value independently, a CNN works with small local regions of the image and learns filters that can detect useful patterns. These filters slide across the input, allowing the network to find the same pattern in different locations. This idea of local connections and shared weights is one of the core principles of CNNs, along with pooling and stacking multiple layers to build deeper feature representations Lecun et al. (2015)

A typical CNN architecture, like the one shown in Figure 2.3, begins with convolutional layers that extract simple features such as edges or textures. These

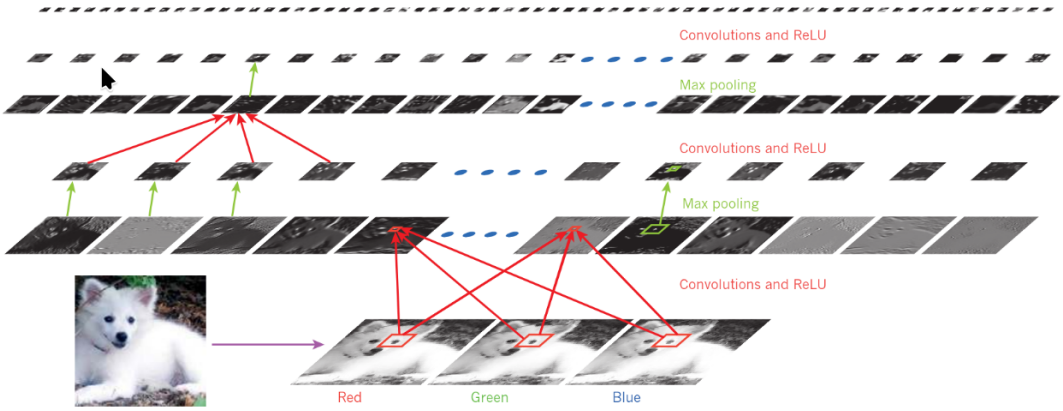

Figure 2.3: Inside a convolutional network

are followed by pooling layers, which reduce spatial resolution and make the representation more robust to small shifts in the input. By stacking several convolution–activation–pooling stages, the network gradually learns more abstract features: edges combine into motifs, motifs into shapes, and shapes into full objects. Finally, fully-connected layers interpret these features to produce the final output. This hierarchical processing makes CNNs especially powerful for visual tasks.

This ability to automatically extract meaningful features from raw images is exactly why CNNs became so important in reinforcement learning. Instead of trying to manually define state features—as classical Q-learning would require—a CNN can learn them directly from the raw game frames.

2.6.3 DQN

DeepMind were first who combined CNN and Q-learning algorithms, which made revolution in Reinforcment learning as their agent was first to learn directly from high-dimensional sensory input.

The CNN works by learning visual features automatically through repeated patterns - convolutional layers start with basic features like edges, then stack to detect increasingly complex objects. Starting from the left, raw game frames pass through these convolutional layers that extract visual features like edges and objects. They feed into fully connected layers that integrate spatial information,

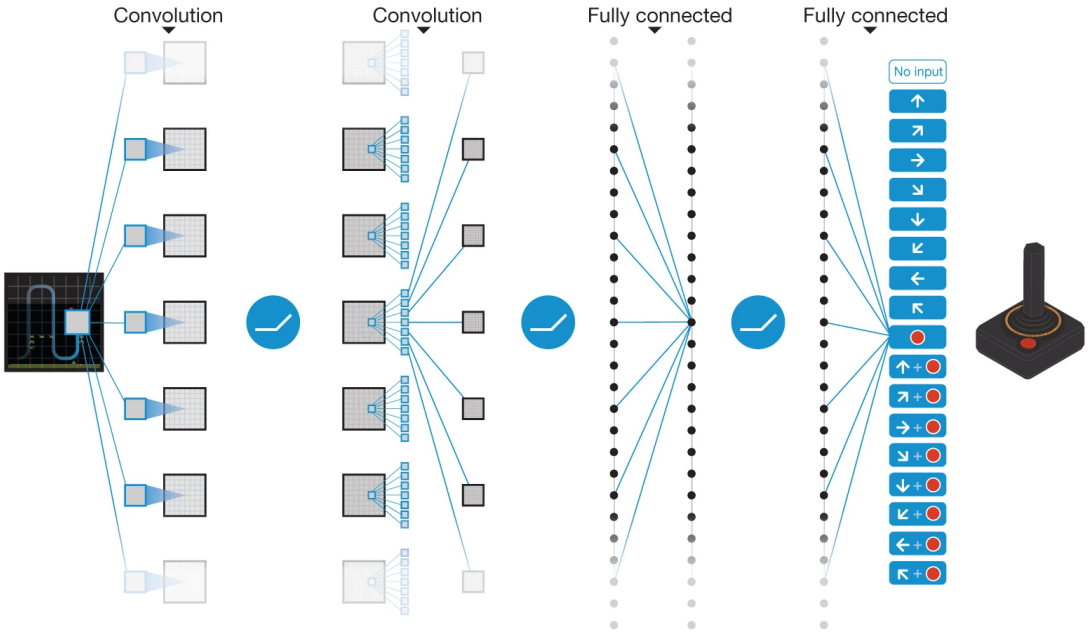

Figure 2.4: DQN Architecture on Atari Breakout Mnih et al. (2015)

and Q-values for each action emerge on the right. DQN also stabilized training with two key techniques: experience replay, which stores and randomly samples past experiences to break correlations, and a target network that updates slowly to provide stable learning targets. These innovations achieved human-level performance across 49 Atari games with a single algorithm.

Lets go through it in more details now.

In the original DeepMind system which is shown in Figure 2.4 the input image (screenshot of the Atary game) consisted of 4 grayscale game frames staked together with dimensions of 84×84 pixels. This game agent knowledge not only of object positions but their velocity and motion directly without need to hand-craft this features. Mnih et al. (2015)

That input frames are passing to CNN network, first layer applies 32 filters(kernels) of size 8×8 with stride size of 4, following by ReLu. Then on the third layer network applies 64 filters of size 4×4 with stride size of 2, following again by ReLu. The third layer applies another 64 filters but with cubic size of 3 this time and stride of 1. As a result we are getting mashed mozaic something which is flattened and fet into a fully–connected layer of 512 units, and finally

into an output layer containing one Q-value for each possible action in the game.

Training DQN uses a modified Q-learning loss function:

$$L_i = \mathbb{E}_{(s,a,r,s')\sim U(D)} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right],$$

where θ are the network parameters and θ^- are the parameters of the target network.

To make Q-learning stable with deep networks, DeepMind introduced two key innovations. The first is *experience replay*, which stores the agent's past transitions and samples them uniformly at random. This breaks the strong correlations between consecutive game frames and allows the network to be trained using ***minibatches***, improving stability and data efficiency. The second innovation is the use of a *target network* whose parameters θ^- are updated only every C steps. This delays the bootstrapping updates and prevents harmful feedback loops that would otherwise cause divergence when the network tries to predict its own moving targets.

With these components combined, DQN became the first reinforcement learning agent to achieve human-level performance across 49 Atari 2600 games using a single neural network architecture and minimal prior knowledge. This result demonstrated that combining deep learning with reinforcement learning enables agents to learn effective policies directly from high-dimensional sensory input.

2.7 Policy Gradients and Actor-Critic Methods

All algorithms discussed above had something in common: they were value-based methods. They learned policy indirectly; for example, in DQN, the policy was derived from the values: we needed to calculate Q-values and then pick the largest one or do so at random. There is another group of algorithms that have a different strategy – Policy Gradient.

These algorithms learn policy directly by parameterizing it; mathematicians use the symbol θ to denote it. If in previous algorithms, policy was marked as $\pi(s|a)$, and we needed an additional formula to predict best actions, in policy

gradient policy with new parameter theta would be our formula of prediction and learning by itself, and it looks like this:

$$\pi(a|s,\theta) = \Pr\{A_t = a|S_t = s, \theta_t = \theta\}$$

To be clearer, we have seen θ in formulas before, including in DQN.

$$L_i = \mathbb{E}_{(s,a,r,s')\sim U(D)}\left[\left(r + \gamma \max_{a'} Q(s',a';\theta^-) - Q(s,a;\theta)\right)^2\right],$$

But it's different. In DQN, θ represented the network's weights that estimated the score (Q-value) of an action, whereas in Policy Gradient, theta represented the network's weights that directly determined the probability of an action. With Policy gradient, we don't need argmax anymore, because the network doesn't yield scores; it gives you the probability of each possible action, and we just sample from that distribution.

By knowing the probabilities of each state in the environment, all that is needed for training is a slight nudge to parameter theta at each step, so the model can understand in which direction it should shift the probabilities. Unlike DQN, we don't need to know true values; all that matters is shifting theta based on the reward returned by the environment: higher reward -¿ higher shift in theta, to increase the probability of taking the same action next time.

It's important to note that to make the method work faster and more accurately, the critical part of this process is comparing of a reward agains a baseline; otherwise, if the agent were in a situation where actions draw a different reward, but all of them are positive, it would significantly slow down learning if there were no baseline. Policy gradient not just asks the question "did we get a reward?", but "did we get a better reward than usual?" instead. Baseline works by comparing the return (G_t) to a "Baseline" ($V(s)$). the formula also known as Policy Gradient Theorem:

$$\theta_{t+1} = \theta_t + \alpha(G_t - b(S_t))\nabla \ln \pi(A_t|S_t,\theta_t)$$

Guarantees that we only need to compute the gradient of our own policy, which is fully known and differentiable, without needing to model the environment's complex dynamics.

2.7.1 REINFORCE Algorithm

The simplest implementation of Policy Gradient methods is REINFORCE, presented by Williams in 1988, which established a mathematical foundation for policy gradient methods Williams (a). The name is an acronym for "REward Increment = Nonnegative Factor × Offset Reinforcement × Characteristic Eligibility," which describes the form of the algorithm.

REINFORCE is a method used in reinforcement learning to improve how decisions are made. It learns by trying actions and then adjusting the chances of those actions based on the total reward received afterwards. Unlike other methods that estimate how good each action is, REINFORCE directly learns the best way to choose actions. This makes it useful for tasks where there are many possible actions or continuous choices and when it is hard to estimate the value of each action.

The core challenge REINFORCE addresses is that the environment is not differentiable. In reinforcement learning, an agent interacts with an environment that can be viewed as a black box: it processes an action and produces the next state and a reward. Since we cannot compute derivatives through the environment, we cannot backpropagate through it. The agent contains a neural network with learnable weights, and to update these weights, gradients are required. Because gradients through the environment are unavailable, we must find alternative ways to perform learning.

An intuitive analogy is a dog receiving treats. If an action leads to a good outcome, that action should be taken more often in the future. If one action produces a large reward, the probability distribution over actions should be adjusted so that this action becomes more likely while others become less likely. More generally, reward is used as a learning signal. When differentiation through the process that consumes network outputs is impossible, behavior is adjusted directly in the direction of reward. This is the core idea of REINFORCE.

Formally, consider a network facing an associative immediate-reinforcement learning task. Suppose that the learning algorithm for this network is such that at the end of each trial each parameter w_{ij} in the network is incremented by an amount

$$\Delta w_{ij} = \alpha_{ij}(r - b_{ij})e_{ij},$$

where α_{ij} is a learning rate factor, b_{ij} is a reinforcement baseline, and

$$e_{ij} = \frac{\partial \ln g_i}{\partial w_{ij}}$$

is called the characteristic eligibility of w_{ij}. Suppose further that the reinforcement baseline b_{ij} is conditionally independent of y_i, given W and x_i, and the rate factor α_{ij} is nonnegative and depends at most on w_{ij} and t. Any learning algorithm having this particular form will be called a REINFORCE algorithm Williams (b).

In practice, REINFORCE operates on a policy $\pi_\theta(a|s)$ parameterized by θ, which outputs an action given a state. The algorithm proceeds as follows. Initialize the policy parameters and step size α. Then repeat:

1. Generate a full episode by following the policy:

   $(s_0, a_0, r_1, s_1, \ldots, s_T)$.

2. For each timestep t, compute the discounted return:

$$G_t = \sum_{k=0}^{T-t} \gamma^k r_{t+k+1}.$$

3. Update parameters:

$$\theta \leftarrow \theta + \alpha\, G_t \nabla_\theta \log \pi_\theta(a_t|s_t).$$

Actions are obtained by sampling from the policy distribution, typically produced via a softmax over network outputs. After sampling, the selected action is evaluated by computing its log-probability under the same policy. The loss is formed as:

$$L = -G_t \log \pi_\theta(a_t|s_t).$$

The negative sign is required because optimization minimizes loss while REINFORCE seeks to maximize expected return. REINFORCE uses reward as a surrogate gradient, allowing policy learning even when the environment is non-differentiable by directly increasing the probability of actions that lead to favorable outcomes.

REINFORCE algorithms are useful in their own right and, perhaps more importantly, may serve as a sound basis for developing other more effective reinforcement learning algorithms. One major advantage of the REINFORCE approach is that it represents a prescription for devising statistical gradient-following algorithms for reinforcement-learning networks of units that compute their random output in essentially any arbitrary fashion. Also, because it is a gradient-based approach, it integrates well with other gradient computation techniques such as backpropagation. The main disadvantages are the lack of a general convergence theory applicable to this class of algorithms and, as with all gradient algorithms, an apparent susceptibility to convergence to false optima Williams (b).

2.7.2 REINFORCE with Baseline

One limitation of the basic REINFORCE algorithm is that it can exhibit unbounded behaviour Phansalkar et al.. One improvement to address this is the introduction of a baseline. The policy gradient theorem can be generalized to include a comparison of the action value to an arbitrary baseline $b(s)$.

The baseline can be any function, even a random variable, as long as it does not vary with a; the equation remains valid because the subtracted quantity is zero. The update rule that we end up with is a new version of REINFORCE that includes a general baseline:

$$\theta \leftarrow \theta + \alpha_\theta (G_t - b(S_t)) \nabla_\theta \log \pi(A_t | S_t, \theta).$$

Because the baseline could be uniformly zero, this update is a strict generalization of REINFORCE. In general, the baseline leaves the expected value of the update unchanged, but it can have a large effect on its variance. This effect is demonstrated in Figure 2.5, where adding a baseline significantly accelerates learning compared to plain REINFORCE.

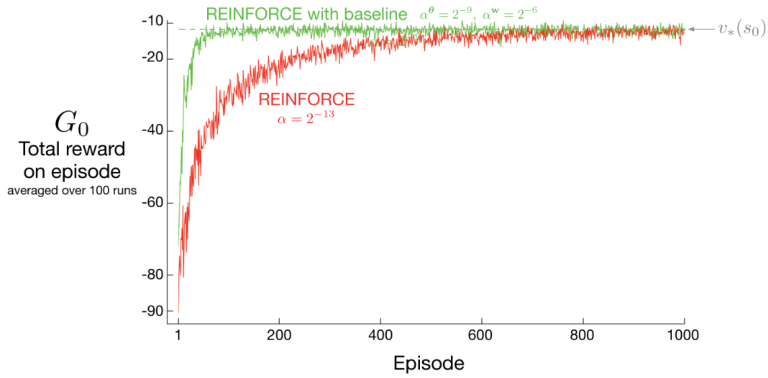

Figure 2.5: Adding a baseline to REINFORCE can make it learn much faster. The step size used for plain REINFORCE is that at which it performs best. Source: Sutton and Barto (2018).

For Markov decision processes, the baseline should vary with state. In some states all actions have high values and a high baseline is needed to differentiate the higher valued actions from the less highly valued ones; in other states all actions have low values and a low baseline is appropriate.

One natural choice for the baseline is an estimate of the state value, $\hat{v}(S_t, w)$, where $w \in \mathbb{R}^m$ is a weight vector learned by one of the methods presented previously. Because REINFORCE is a Monte Carlo method for learning the policy parameters θ, it is natural to also use a Monte Carlo method to learn the state-value weights w.

2.7.2.1 Episodic REINFORCE with Baseline

Input: a differentiable policy parameterization $\pi(a|s, \theta)$

Input: a differentiable state-value function parameterization $\hat{v}(s, w)$

Algorithm parameters: step sizes $\alpha_\theta > 0$, $\alpha_w > 0$

Initialize policy parameter $\theta \in \mathbb{R}^{d'}$ and state-value weights $w \in \mathbb{R}^d$

For each episode:

1. Generate an episode $(S_0, A_0, R_1, \ldots, S_T)$ following $\pi(\cdot|\cdot, \theta)$

2. For each timestep $t = 0, 1, \ldots, T-1$:

(a) Compute return:

$$G_t = \sum_{k=t+1}^{T} \gamma^{k-t-1} R_k$$

(b) Update state-value weights:

$$w \leftarrow w + \alpha_w (G_t - \hat{v}(S_t, w)) \nabla_w \hat{v}(S_t, w)$$

(c) Update policy parameters:

$$\theta \leftarrow \theta + \alpha_\theta (G_t - \hat{v}(S_t, w)) \nabla_\theta \log \pi(A_t | S_t, \theta)$$

This algorithm has two step sizes, α_θ and α_w. Choosing the step size for the value function is relatively straightforward. It is less clear how to set the step size for the policy parameters, whose best value depends on the range of variation of rewards and on the policy parameterization Sutton and Barto (2018).

Although the REINFORCE-with-baseline method learns both a policy and a state-value function, we do not consider it to be an actor–critic method because its state-value function is used only as a baseline, not as a critic. That is, it is not used for bootstrapping (updating the value estimate for a state from the estimated values of subsequent states), but only as a baseline for the state whose estimate is being updated Sutton and Barto (2018).

2.7.3 Actor – Critic Methods

Actor-Critic methods arise from policy gradient in the same way that TD learning emerged from Monte Carlo. Actor-Critic implements bootstrapping to solve the same waiting problem. In the original Policy Gradient (like Monte Carlo methods), we must wait until the full episode ends before updating our policy. This makes training slow and produces high-variance gradients. To mitigate this, instead of using actual returns, we estimate them.

In original policy gradient, the baseline b(s) could be any function—even just an average reward. But by making the baseline a learned value function ($b(s) = V(s)$), we effectively have two networks: one that picks actions (the policy π) called the Actor, and one that judges states (the value V) called the Critic.

The actor 'performs'—deciding what action to take according to $\pi(a|s, \theta)$—while the critic 'criticizes' those actions, telling the actor whether a state is good or bad through $V(s, w)$. Note that these networks have separate parameters:θ for the actor and W for the critic.

The critic enables bootstrapping. Instead of waiting for the full return G_t, we can estimate the advantage using the TD error:

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

This quantity estimates how much better the action taken was compared to the average action in that state.

To train both networks at the same time, we perform two updates at each time step:

Critic update: adjust w to make V(s) more accurate (same as TD learning from Section 2.5.3):

$$w \leftarrow w + \alpha_w \delta_t \nabla V(S_t, w)$$

Actor update: adjust θ using the critic's judgment:

$$\theta \leftarrow \theta + \alpha_\theta \delta_t \nabla \ln \pi(A_t|S_t, \theta)$$

Note that δ_t appears in both formulas—the critic's judgment guides everything, and this is how the two networks are connected.

2.7.3.1 Advantages of Actor-Critic

Actor-Critic methods combine the strengths of both value-based and policy-based approaches. Because they are built on policy gradients, they can learn stochastic policies, which is useful in environments requiring randomness (such as games with incomplete information). DQN, in contrast, is deterministic—it always selects the action with the maximum Q-value. Additionally, actor-critic methods can handle continuous action spaces (such as controlling a steering angle), whereas value-based methods require computing argmax over all possible actions, which is infeasible when actions are continuous.

Compared to pure policy gradient methods, actor-critic has lower variance because it uses TD estimates rather than waiting for full episode returns. This also means updates happen at every time step, enabling faster, online learning.

2.7.4 Advantage Actor-Critic (A2C)

Advantage Actor-Critic (A2C) is a synchronous variant of the Asynchronous Advantage Actor-Critic (A3C) method (Mnih et al.), using the advantage function $A(s, a) = Q(s, a) - V(s)$ as the policy gradient signal to reduce variance while maintaining unbiased updates. Unlike A3C, which runs multiple actors in parallel threads with asynchronous gradient updates, A2C collects experience from all parallel environments synchronously before performing a single update, making it simpler to implement and reason about. TODO: Expand this subsection with full description of A2C, advantage estimation, generalised advantage estimation (GAE), and comparison with A3C.

2.7.5 Proximal Policy Optimisation (PPO)

Proximal Policy Optimisation (Schulman et al.) is an actor-critic method that improves on A2C and REINFORCE by using a clipped surrogate objective to prevent excessively large policy updates, achieving stable training without the complexity of second-order trust-region methods (such as TRPO). PPO is currently one of the most widely used deep RL algorithms and serves as the primary actor-critic representative in the experiments of this thesis. TODO: Expand this subsection with full description of the PPO clipped objective, generalised advantage estimation, epoch-level minibatch updates, and key hyperparameters.

2.8 Experimental Frameworks and Tooling

2.8.1 Gymnasium

Gymnasium (Towers et al.) is a Python library providing a standardised interface for reinforcement learning environments. It is the maintained successor to

OpenAI Gym and is now the de-facto standard for RL research and benchmarking. Gymnasium defines a consistent `reset/step/render` API that decouples algorithm implementations from environment details, making it straightforward to swap environments without changing training code. It ships with a wide range of reference environments, including the classic-control tasks (CartPole, Acrobot, LunarLander) used in Experiment 1 and, via the Arcade Learning Environment (Bellemare et al.), the Atari 2600 games used in Experiments 2 and 3. All environments in this thesis are instantiated through the Gymnasium API.

2.8.2 Stable-Baselines3

Stable-Baselines3 (SB3, v2.7.1) (Raffin et al.) is a set of reliable, well-tested PyTorch implementations of standard deep RL algorithms, including DQN, PPO, and A2C. It provides a consistent training API, built-in vectorised environments, evaluation utilities, and callback infrastructure. A key design goal of SB3 is reproducibility: all algorithms ship with carefully chosen default hyperparameters that are documented and stable across releases. This project uses SB3 exclusively for all algorithm implementations. Using a single, shared framework ensures that observed performance differences reflect genuine algorithmic properties rather than implementation quality or incidental hyperparameter choices.

2.8.3 Stable-Baselines3 Contrib and Extensions

`sb3-contrib` is the official extension package for SB3, hosting algorithms that are functional but not yet part of the main stable release. Two algorithms used in Experiment 1 are sourced from this package: QR-DQN, a distributional variant of DQN that models the full return distribution rather than a scalar Q-value, and RecurrentPPO, a variant of PPO augmented with an LSTM layer to handle partial observability. Both are used with their `sb3-contrib` default hyperparameters under the same fair-comparison protocol as the main SB3 algorithms.

2.8.4 Reproducibility Considerations

Statistical evaluation in this thesis follows the `rliable` methodology of Agarwal et al., which provides robust aggregate metrics (IQM, optimality gap, performance profiles, and probability of improvement) with stratified bootstrap confidence intervals. These metrics are more reliable than mean scores in the small-sample regime typical of RL research. The full evaluation protocol, including seeding strategy and score normalisation, is defined in Section 2.9.

2.9 Evaluation Framework

This section defines what data are produced by reinforcement learning training, how evaluation scores are extracted from that data, and what metrics are used to compare algorithms throughout this thesis. Because this study uses a small set of $M \leq 3$ environments, *per-environment* results are the primary evidence, and cross-environment aggregates are treated as descriptive summaries of the chosen tasks rather than universal generalisations (Agarwal et al.; Patterson et al.).

Seeding strategy. To ensure reproducibility while avoiding correlations induced by sequential seeding, all experimental runs were initialised from a single *master seed*. Individual run seeds were deterministically derived using NumPy's `SeedSequence` spawning mechanism, which avoids linear relationships between seeds that may introduce inter-stream correlations in certain pseudorandom number generators (Matsumoto et al.). All randomness sources (environment, NumPy, and PyTorch) were seeded using the derived run-specific seeds.

2.9.1 Data Produced by RL Interaction

During training, at each time step k the agent–environment interaction yields the transition tuple $(s_k, a_k, r_k, s_{k+1}, d_k)$, where s_k is the state, a_k the action, r_k the scalar reward, s_{k+1} the next state, and $d_k \in \{0, 1\}$ a terminal indicator. Algorithm comparisons are not made on training rewards directly, because training behaviour (exploration noise, replay buffer dynamics, on- vs off-policy sampling) is algorithm-dependent and not a stable measure of generalisation. Instead, a

fixed **evaluation protocol** is used under which every algorithm is assessed identically.

2.9.2 Evaluation Scores: Definitions and Notation

An evaluation episode has an (environment-defined) length H, and the **episodic return** is $G = \sum_{k=0}^{H-1} r_k$. I report undiscounted G throughout. Index conventions: m over environments, n over seeds, t over evaluation checkpoints, T the final checkpoint, E the number of evaluation episodes. At each checkpoint t, the mean episodic return over E episodes is:

$$x_{m,n}(t) \;=\; \frac{1}{E}\sum_{e=1}^{E} G_{m,n}^{(e)}(t).$$

These values form the **score matrix** $X(t) \in \mathbb{R}^{N\times M}$, with rows indexing seeds and columns indexing environments. All metrics below are computed from $X(T)$ at the final checkpoint.

2.9.3 Uncertainty Quantification: Stratified Bootstrap

Reinforcement learning exhibits high variance across independent training runs. To quantify statistical uncertainty, **95% stratified bootstrap confidence intervals** are reported for all non-point-estimate metrics. For each environment m, seeds are resampled with replacement; the statistic is recomputed on each replicate and the 2.5% and 97.5% percentile bounds are taken. This procedure is recommended for the few-run regime (Agarwal et al.).

2.9.4 Per-Environment Metrics (Primary)

Per-environment metrics operate on $\{x_{m,n}(t)\}_{n=1}^{N}$ for a fixed environment m:

- **Learning curve**: central tendency (mean or median over seeds) of $x_{m,n}(t)$ against training budget t, with pointwise uncertainty bands. Reveals learning dynamics that a single final number cannot capture.

- **Final performance at fixed budget** T: central tendency of $\{x_{m,n}(T)\}$ with 95% bootstrap CI. Answers "how good is the agent after the same amount of experience?" (Agarwal et al.)
- **Sample efficiency**: normalised area under the learning curve, $\mathrm{AUC}_{m,n} = \sum_j x_{m,n}(t_j)(t_j - t_{j-1})/T$. Higher AUC indicates faster learning under the same budget.
- **Reliability**: interquartile range (IQR) of $\{x_{m,n}(T)\}_n$ captures robustness across seeds (Chan et al.).

2.9.5 Cross-Environment Aggregate Metrics (Secondary)

Cross-environment metrics summarise the score matrix $X(T)$. With $M \leq 3$ environments, these are descriptive summaries rather than claims of general performance (Agarwal et al.; Patterson et al.):

- **Interquartile mean (IQM)**: pool all MN values, trim the lowest and highest 25%, average the middle 50%. Robust to outliers and often yields tighter uncertainty in the few-run regime (Agarwal et al.).
- **Probability of improvement (POI)**: $P(A > B) = \frac{1}{M}\sum_m \Pr(x^A_{m,\cdot} > x^B_{m,\cdot})$. Interpretable probability that algorithm A outperforms B on a randomly chosen seed and environment.
- **Performance profile**: fraction of runs $\hat{F}(\tau) = \frac{1}{MN}\sum_{m,n} \mathbb{I}[x_{m,n}(T) \geq \tau]$ at each threshold τ. Visualises the full spectrum of outcomes.
- **Optimality gap**: $\mathrm{Gap}(\tau) = \tau - \frac{1}{MN}\sum_{m,n} \min(x_{m,n}(T), \tau)$. Measures shortfall from a predefined target while clipping large outliers.

2.10 Conclusion

This chapter provided a high-level coverage of the basics of Reinforcement Learning, and should be sufficient for any reader to understand the idea of further implementations. It began with an overview of artificial intelligence and machine

learning, then moved on to the core concepts of RL: MDPs, the Bellman equation, and exploration versus exploitation. Value-based methods came next—Dynamic Programming, Monte Carlo, TD Learning, and DQN. Finally, Policy, REINFORCE and Actor-Critic methods showed a different path: learning policy directly rather than through value estimation.

Chapter 3

Evaluation

this summary no longer true This chapter restates the research question, synthesises the findings from all three experiments, provides a direct answer to the research question, and then discusses the limitations of the work and directions for future research.

3.1 Research Question

The thesis was organized around a single research question:

> *"How does the performance of value-based Deep Q-Network agents compare to actor-critic agents (specifically PPO and A2C) in discrete-action environments, including their sample efficiency and training consistency?"*

Three experiments were designed to answer this question progressively: Experiment 1 on classic-control environments with MLP policies, Experiment 2 on Atari with CNN policies at a 5M-step budget, and Experiment 3 extending Atari to 20M steps across three environments. Each experiment produced per-environment results and cross-environment aggregates using the rliable evaluation framework described in Section 2.9.

3.2 Synthesis of Results

3.2.1 Experiment 1: Classic-Control Benchmark

On CartPole-v1, Acrobot-v1, and LunarLander-v3, PPO and A2C decisively outperform the DQN family. PPO achieves a normalised IQM of 0.981 across the three environments, with probabilities of improvement over every other algorithm of at least 0.89. The DQN family (DQN and QR-DQN) fails to learn at the allowed training budget (200K steps for CartPole and Acrobot, 500K for LunarLander), scoring at or near random across all seeds. RPPO collapses on Acrobot in a way that suggests its recurrent policy is ill-suited to fully-observable MDPs. Classic-control is not a DQN-friendly setting at these budgets.

3.2.2 Experiment 2: Atari Benchmark

Moving to Atari with CNN policies at 5M environment steps per seed partially flips the per-environment ranking. On Pong, PPO continues to dominate (mean return ≈ 20 vs DQN ≈ -8). On Breakout, DQN slightly outperforms PPO, though both remain well below 11% of the theoretical maximum. The cross-environment aggregate still favours PPO ($\text{IQM}_{\text{PPO}} = 0.529$ vs $\text{IQM}_{\text{DQN}} = 0.130$, $P(\text{PPO} > \text{DQN}) = 0.648$), but the margin is much smaller than in Experiment 1, and the $M = 2$ aggregate is visibly dominated by Pong's dynamic range.

3.2.3 Experiment 3: Extended Atari Benchmark

Extending the budget to 20M steps and adding Seaquest as a third Atari environment produces the most interesting result of the thesis: the cross-environment aggregate *flips*. The IQM now favours DQN (0.443 vs 0.277), driven by DQN's 3.5× Breakout advantage at the larger budget. The pairwise probability of improvement, however, still favours PPO ($P(\text{DQN} > \text{PPO}) = 0.393$). Per-environment, DQN wins Breakout decisively (229.07 vs 65.50), PPO wins Pong (19.78 vs 11.20) and Seaquest (1173.92 vs 714.00) moderately.

The practical takeaway is that the IQM/POI disagreement is itself a finding: with $M = 3$ environments and 10 seeds, the choice of cross-environment aggre-

gation method can flip the answer to "which algorithm is better". Experiment 3 also established that the training pipeline is bitwise deterministic on the target RTX 4090 hardware — a practical result for anyone attempting to reproduce this work.

3.3 Answer to the Research Question

Neither family dominates. The correct comparison is per-environment, and even within Atari the ranking depends on the environment, the training budget, and the aggregation method:

- On dense-reward, low-dimensional, fully-observable classic control, PPO and A2C are clearly stronger than DQN and QR-DQN at the budgets tested.
- On Atari Pong, PPO is consistently stronger at both 5M and 20M step budgets.
- On Atari Breakout, DQN edges PPO at 5M and wins decisively at 20M.
- On Atari Seaquest at 20M, PPO wins moderately.
- Cross-environment aggregates are fragile at the M values used here, and the choice of IQM vs pairwise POI can flip the aggregate answer. Per-environment reporting is the primary evidence.

The honest summary is that PPO is the stronger default — it performs well across all settings tested and dominates completely on classic control — but DQN closes the gap at longer Atari budgets and outperforms PPO outright on Breakout. Both algorithms exist for good reason, and the choice should be made based on the specific environment and budget, not convention.

3.4 Threats to Validity

- **Default hyperparameters throughout.** No per-algorithm or per-environment tuning was performed. This keeps the comparison fair in the sense that no

algorithm was given an advantage, but it means the numbers reflect "out-of-the-box SB3" rather than "best achievable". The gap between the two may be large, especially for DQN on Atari.

- **Small number of environments (M).** Classic control uses $M = 3$, Atari uses $M = 2$ (Experiment 2) then $M = 3$ (Experiment 3). These are well below the $M = 50+$ used in standard RL benchmark papers. The aggregate metrics are correspondingly fragile: adding or removing one environment can change the cross-env aggregate substantially.

- **Fixed training budgets.** 200K–500K steps on classic control, 5M on Atari-Exp-2, 20M on Atari-Exp-3. These are smaller than the original DQN and PPO paper budgets (50M frames) and almost certainly disadvantage DQN more than PPO, since DQN's replay buffer requires more experience to warm up.

- **Experiment 3 buffer size oversight.** The intended DQN replay buffer size was 400K, but the experiments ran with 100K (Stable-Baselines3 default). The results are internally consistent and the determinism finding confirms exact reproducibility, but the Breakout lead attributed to DQN may look different with the intended buffer.

- **Timing data incomplete for Experiment 3 PPO.** Evaluation scores are complete for all seeds; only per-seed wall-clock timing has gaps for the PPO Pong runs. The timing section in Chapter **??** uses a placeholder pending completion of the retraining.

Chapter 4

Discussions & Conclusions

Add a new Chapter 8: "Discussion and Conclusions" — approximately one page, free prose (no bullet lists), structured as three paragraphs: Paragraph 1: restate the research question and give the direct answer. Paragraph 2: reflect on the difficulty of the project and the key learnings. Paragraph 3: future work (moved from Chapter 7).

4.1 Future Work

- **Repeat Experiment 3 with the intended 400K DQN buffer size.** This is the most direct follow-up: the buffer oversight means the Breakout result should be re-verified with the correct configuration.

- **Longer training budgets.** Running to 50M+ frames on Atari would bring the comparison closer to the original DQN and PPO paper settings, and may further change the per-environment ranking.

- **Larger M.** Adding more Atari environments (SpaceInvaders, MsPacman, BeamRider) and more classic-control environments (MountainCar, Pendulum) would make the cross-environment aggregates more stable and the conclusions more generalisable.

- **Hyperparameter tuning study.** Tuning one environment per algorithm would quantify the gap between "default SB3" and "best achievable", giving

context to the default-hyperparameter numbers reported here.

- **Continuous-control extension.** A MuJoCo or continuous-control study would test whether the per-environment pattern seen here generalises beyond discrete-action domains, where policy-gradient methods are typically expected to have a larger advantage.

References

Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems*. doi: 10.48550/arXiv.2108.13264. URL `https://arxiv.org/abs/2108.13264`. NeurIPS 2021. Proceedings record: 10.5555/3540261.3542505. 7, 37, 38, 39

Andrew G. Barto, Richard S. Sutton, and Christopher J. C. H. Watkins. Learning and sequential decision making. Technical report, University of Massachusetts Amherst, Amherst, MA, 1989. 2, 13

Marc G. Bellemare, Yavar Naddaf, Joel Veness, and Michael Bowling. The arcade learning environment: An evaluation platform for general agents. 47:253–279. doi: 10.1613/jair.3912. 36

Apostolos N. Burnetas and Michael N. Katehakis. Optimal adaptive policies for markov decision processes. *Mathematics of Operations Research*, 22(1):222–255, 1997. doi: 10.1287/moor.22.1.222. 14, 16

Stephanie C. Y. Chan, Samuel Fishman, John Canny, Anoop Korattikara, and Sergio Guadarrama. Measuring the reliability of reinforcement learning algorithms. In *International Conference on Learning Representations*. doi: 10.48550/arXiv.1912.05663. URL `https://arxiv.org/abs/1912.05663`. 39

CNN. Ai set to exceed human brain power. `https://edition.cnn.com/2006/TECH/science/07/26/ai.future/`, 2006. Accessed: 2006-07-26. 12

Daniel Crevier. *AI: The Tumultuous History of the Search for Artificial Intelligence*. Basic Books, New York, 1993. 12

Neil De La Fuente and Daniel A. Vidal Guerra. A comparative study of deep reinforcement learning models: DQN vs PPO vs A2C. 5

REFERENCES

Sharon Goldman. Deep learning breakthroughs that reshaped ai. *MIT Technology Review*, 2022. 12

Aurélien Géron. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. O'Reilly Media, Incorporated, Sebastopol, 2022. Accessed November 20, 2025. ProQuest Ebook Central. 24

Leslie Pack Kaelbling, Michael L. Littman, and Andrew W. Moore. Reinforcement learning: A survey. *Journal of Artificial Intelligence Research*, 4:237–285, 1996. doi: 10.1613/jair.301. 14

Andreas Kaplan and Michael Haenlein. Siri, siri, in my hand: Who's the fairest in the land? on the interpretations, illustrations, and implications of artificial intelligence. *Business Horizons*, 62(1):15–25, 2019. doi: 10.1016/j.bushor.2018.08.004. 12

Yann Lecun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521 (7553):436–444, May 2015. doi: 10.1038/nature14539. URL `https://hal.science/hal-04206682`. 3, 24

Shengbo Li. *Reinforcement Learning for Sequential Decision and Optimal Control*. Springer Verlag, Singapore, 1st edition, 2023. ISBN 978-9-811-97783-1. doi: 10.1007/978-981-19-7784-8. 3

Makoto Matsumoto, Isaku Wada, Ai Kuramoto, and Hyo Ashihara. Common defects in initialization of pseudorandom number generators. 17(4):15–es. ISSN 1049-3301. doi: 10.1145/1276927.1276928. URL `https://dl.acm.org/doi/10.1145/1276927.1276928`. 37

B. Matzliach, I. Ben-Gal, and E. Kagan. Detection of static and mobile targets by an autonomous agent with deep q-learning abilities. *Entropy*, 24(8):1168, 2022. ISSN 1099-4300. doi: 10.3390/e24081168. 3

McKinsey & Company. Notes from the ai frontier: Modeling the impact of ai on the world economy. `https://www.mckinsey.com/featured-insights/artificial-intelligence/notes-from-the-ai-frontier-modeling-the-impact-of-ai-on-the-world-economy`, 2018. 12

Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous

methods for deep reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning*, volume 48 of *PMLR*, pages 1928–1937. URL `http://proceedings.mlr.press/v48/mniha16.html`. 35

Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518 (7540):529–533, 2015. doi: 10.1038/nature14236. URL `https://doi.org/10.1038/nature14236`. Issue date: 26 February 2015. vii, 6, 24, 26

National Research Council. *Funding a Revolution: Government Support for Computing Research*. National Academies Press, Washington, DC, 1999. 12

Andrew Patterson, Samuel Neumann, Martha White, and Adam White. Empirical design in reinforcement learning. 25(318):1–63. doi: 10.48550/arXiv.2304.01315. URL `https://jmlr.org/papers/v25/23-0183.html`. JMLR version (2024); arXiv preprint: 2304.01315. 37, 39

V.V. Phansalkar, P.S. Sastry, and M.A.L. Thathachar. An optimization approach to the analysis of generalized learning automata algorithms. In *ACE '90. Proceedings of [XVI Annual Convention and Exhibition of the IEEE In India]*, pages 8–11. IEEE. doi: 10.1109/ace.1990.762626. URL `http://dx.doi.org/10.1109/ACE.1990.762626`. 31

Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. 22(268):1–8. URL `http://jmlr.org/papers/v22/20-1364.html`. 36

Stuart J. Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, Hoboken, NJ, 4th edition, 2021. 11, 12

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. doi: 10.48550/arXiv.1707.06347. URL `https://arxiv.org/abs/1707.06347`. 35

Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. Adaptive Computation and Machine Learning Series. The MIT Press, Cambridge,

Massachusetts, second edition, 2018. ISBN 9780262039246. A Bradford Book. vii, 2, 14, 15, 20, 21, 22, 32, 33

Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, Rodrigo Perez-Vicente, Andrea Pierré, Sander Schulhoff, Jun Jet Tai, Hannah Tan, and Omar G. Younis. Gymnasium: A standard interface for reinforcement learning environments. Accepted at NeurIPS Datasets and Benchmarks 2025. 35

unknown. Pac-man game board image. `https://ar.inspiredpencil.com/pictures-2023/pacman-game-board`, 2023. Accessed: November 20, 2025. vii, 24

Ronald J. Williams. On the use of backpropagation in associative reinforcement learning. In *IEEE International Conference on Neural Networks*, pages 263–270. IEEE, a. doi: 10.1109/icnn.1988.23856.

Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. 8(3):229–256, b. ISSN 1573-0565. doi: 10.1007/BF00992696. URL `https://doi.org/10.1007/BF00992696`. 30, 31